

App Dev 1 Project Report

1. Student Details

Name: Vanshita Gautam

Roll Number: 24f1000135

Email: 24f1000135@ds.study.iitm.ac.in

About Me: I am Vanshita, currently enrolled in diploma level BS in Data Science and Applications from IIT Madras, alongside a BA in Psychology, Sociology, English, and Computer Applications from Mahila Mahavidyalaya Degree College, Kanpur. I enjoy working on my projects.

2. Project Details

Project Title: Trekking Management App

Problem Statement:

To build a **Trekking Management Application** web application that allows **Admin, Trek Staff, and Users (Trekksers)** to interact with the system based on their roles.

Approach:

I built this automated, role-based web application using Python, Flask, and SQLAlchemy to completely replace manual coordination and bring administrators, field staff, and trekkers together onto a single, seamless platform.

3. AI/LLM Declaration

I used **Claude** to generate initial boilerplate structures for Flask Blueprints, add flash messages and assist with debugging and defining database relationships.

The extent of AI/LLM usage is around 25-30%, limited to boilerplate code suggestions, UI styling references, and debugging assistance.

All core app workflows, business rules and final integration were executed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
CSS3 & Bootstrap4	Frontend custom styling and bootstrap for flash messages
SQLite	Lightweight local database for storing user data

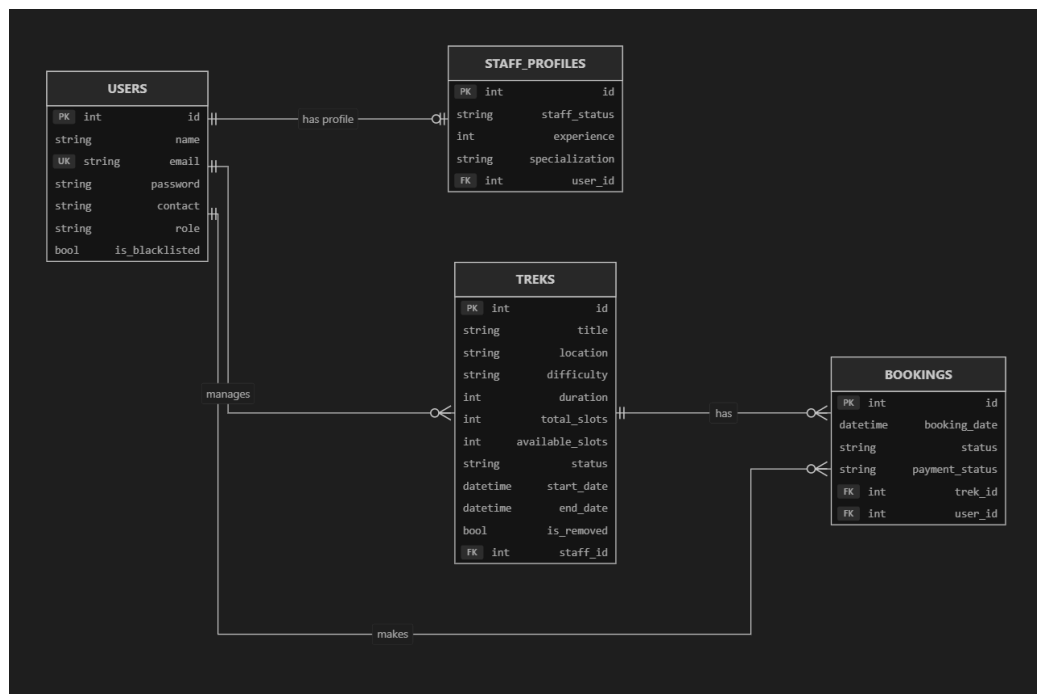
5. Database Schema / ER Diagram

Tables:

1. **User** — stores user profile and authentication details (id, name, email, password, contact, role, is_blacklisted)
2. **StaffProfile** — extended profile for staff members (id, user_id, staff_status, experience, specialisation)
3. **Treks** — trekking event details managed by staff (id, staff_id, title, location, difficulty, duration, total_slots, available_slots, status, start_date, end_date, is_removed)
4. **Bookings** — logs trek reservations by trekkers (id, user_id, trek_id, booking_date, status, payment_status)

Relationships:

- One-to-One → User → StaffProfile
- One-to-Many → User → Trek (staff manages treks)
- One-to-Many → User → Booking (trekker makes bookings)
- One-to-Many → Trek → Booking (trek has many bookings)



7. Architecture and Features

Architecture Overview:

- **app.py** — Entry point — app init, DB setup, Blueprint registration, Admin seed
- **models.py** — SQLAlchemy models — User, StaffProfile, Trek, Booking
- **auth.py** — Blueprint — Login, Register (Trekker & Staff), Logout

- **admin.py** – Blueprint /admin — Trek CRUD, Staff approvals, Trekker management
- **staff.py** – Blueprint /staff — Status/slot updates, participant tracking
- **trekker.py** – Blueprint /trekker — Browsing, booking, payment, cancellation
- **/templates** – Jinja2 templates grouped by role (auth/, admin/, staff/, trekker/)
- **/static** – CSS style.css

Implemented Features:

- Auth — hashed passwords, role-based sessions, before_request guards, blacklist/approval checks
- Admin — full trek CRUD, trekker blacklisting, staff assignment, booking history
- Staff — trek status/slot control, participant management
- Trekker — trek discovery with filters, booking, payment, cancellation, history

8. Video Presentation

Drive Link:

 Video Report final.mp4